Python for Web Development

Lesson 1: Setting Up the Development Environment

Setting Up the Development Environment - Study Notes

Key Concepts

1. **Python installation & version management** – downloading Python, verifying the install, and understanding the role of the `python` vs. `python3` commands.

2. **Virtual environments (`virtualenv` / `venv`)** – isolating project dependencies, creating, activating, and deactivating environments.

3. **VS /Code configuration for Python** – installing the Python extension, selecting the interpreter, and using integrated terminals.

4. **Project structure conventions** – typical folder layout (`src/`, `tests/`, `requirements.txt`, etc.) that supports scalability and collaboration.

5. **Dependency management** – using `pip` and `requirements.txt` (or `pipenv/poetry`) inside a virtual environment.

Summary

Setting up a clean, reproducible development environment is the first step toward successful Python projects. Begin by installing the latest stable release of Python from the official website (or via a package manager). Verify the installation with `python --version` (or `python3`) and ensure the **PATH** variable includes the Python executable so the command works from any terminal.

Next, create an isolated workspace for each project using **virtual environments**. The built in `venv` module or the third party `virtualenv` package lets you keep project specific packages separate from the global interpreter, preventing version conflicts and making deployments predictable. Activate the environment before installing any libraries, and deactivate it when you're done working.

Finally, configure **Visual Studio Code (VS /Code)** as your editor. Install the official Python extension, point it at the interpreter inside your virtual environment, and enable useful features such as linting, IntelliSense, debugging, and test discovery. Adopt a conventional project layout—e.g., a top level `src/` for source code, `tests/` for unit tests, and a `requirements.txt` file for pinned dependencies—to keep the codebase organized and ready for version control.

Important Points

• **Python install**

- Download from <https://www.python.org/downloads/> or use a package manager (`brew`, `apt`, `choco`).

- Check installation: `python --version` (Windows) or `python3 --version` (macOS/Linux).

- Add Python to **PATH** during installation (Windows) or ensure `/usr/local/bin` is in `$PATH` (macOS/Linux).

• **Virtual environment basics**

- Create with the built in module: `python -m venv .venv` (or `python3 -m venv .venv`).
- Activate:
  - Windows: `.\.venv\Scripts\activate`
  - macOS/Linux: `source .venv/bin/activate`
- Deactivate with `deactivate`.
- Packages installed while the env is active go into `.venv/lib/pythonX.Y/site-packages`.
    • **VS /Code setup**
- Install the **Python** extension (Microsoft).
- Open the command palette (`Ctrl+Shift+P` / `Cmd+Shift+P`) !' **Python: Select Interpreter** !' choose the one inside `.venv`.
- Enable linting (e.g., `pylint` or `flake8`) and formatting (e.g., `black`) via settings.
- Use the integrated terminal (``Ctrl+` ``) to run commands inside the activated env.
    • **Typical project structure**

```
my_project/
%
% %  .venv/                # virtual environment (often excluded from VCS)
% %  src/                  # source code
%    % %  __init__.py
%    % %  main.py
% %  tests/                # unit / integration tests
%    % %  test_main.py
% %  requirements.txt      # pinned dependencies
% %  .gitignore            # exclude .venv, __pycache__, etc.
% %  README.md
```

    • **Dependency management**
- Install packages: `pip install <package>` (inside env).
- Freeze exact versions: `pip freeze > requirements.txt`.
- Re install later: `pip install -r requirements.txt`.

Examples

Example 1 – Creating and Using a Virtual Environment

```bash
# 1þ  ã Clone or create a project folder

mkdir demo_app && cd demo_app
# 2þ  ã Create a virtual environment named .venv

python -m venv .venv          # use python3 on macOS/Linux if needed
```

3þ ã Activate it

Windows

.\.venv\Scripts\activate

macOS / Linux

source .venv/bin/activate

4þ ã Install a library (e.g., requests) and freeze dependencies

pip install requests

pip freeze > requirements.txt

5þ ã Verify the library is isolated

python -c "import requests; print(requests.__version__)"

Deactivate when done

deactivate

```

*Explanation:* The steps above create an isolated environment, install a third party package without affecting the system Python, and record the exact version for reproducibility.

**Example 2 – Configuring VS /Code for the Project**

1. Open VS /Code in the project folder: `code .`

2. Press `Ctrl+Shift+P` !' **Python: Select Interpreter** !' choose `.venv\Scripts\python.exe` (Windows) or `.venv/bin/python` (macOS/Linux).

3. Create a `settings.json` (Workspace) to enable linting and formatting:

```json
{
    "python.linting.enabled": true,
    "python.linting.pylintEnabled": true,
    "python.formatting.provider": "black",
    "python.testing.unittestEnabled": true,
    "python.testing.cwd": "${workspaceFolder}"
}
```

4. Open the integrated terminal (`Ctrl+``) – it will start with the virtual environment already activated, allowing you to run `python`, `pip`, or your test suite directly.

Explanation: By pointing VS /Code at the interpreter inside `.venv`, all IntelliSense, linting, and debugging actions use the same isolated packages you installed, guaranteeing consistency between the editor and the command line.
```

Quick Review

1. **Why use a virtual environment for each Python project?**
2. **What command activates a virtual environment on macOS/Linux?**
3. **Which VS /Code extension is essential for Python development?**
4. **Name two typical top level directories in a well structured Python project.**
5. **How do you generate a `requirements.txt` file from the current environment?**

Further Reading

- **Python Packaging Guide** – `pip`, `wheel`, and `setuptools` fundamentals.
- **PEP /508 & PEP /517** – standards for dependency specification and build back ends.
- **Advanced environment tools** – `pipenv`, `poetry`, and `conda` comparisons.
- **Testing frameworks** – `unittest`, `pytest`, and test discovery in VS /Code.
- **Continuous Integration** – automating environment setup with GitHub Actions or GitLab CI.